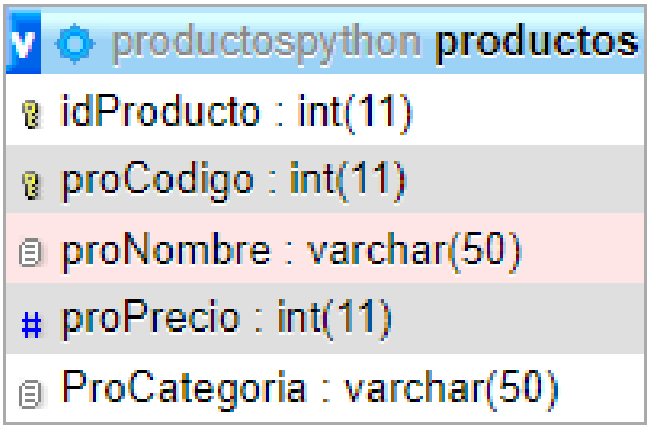
PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
MATERIAL DE APOYO
CONEXIÓN A BASES DE DATOS DESDE PYTHON

Introducción

El presente material de apoyo contiene todo el proceso de conexión a una base de datos **SQLITE** desde **Python** y el proceso de consultas de un **CRUD** a una base de datos.

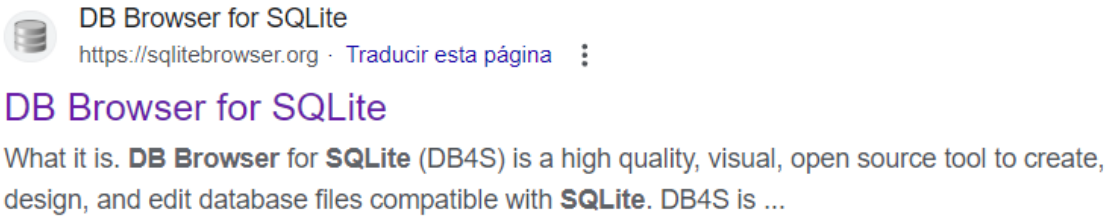
La base de datos a utilizar contiene una sola tabla así:



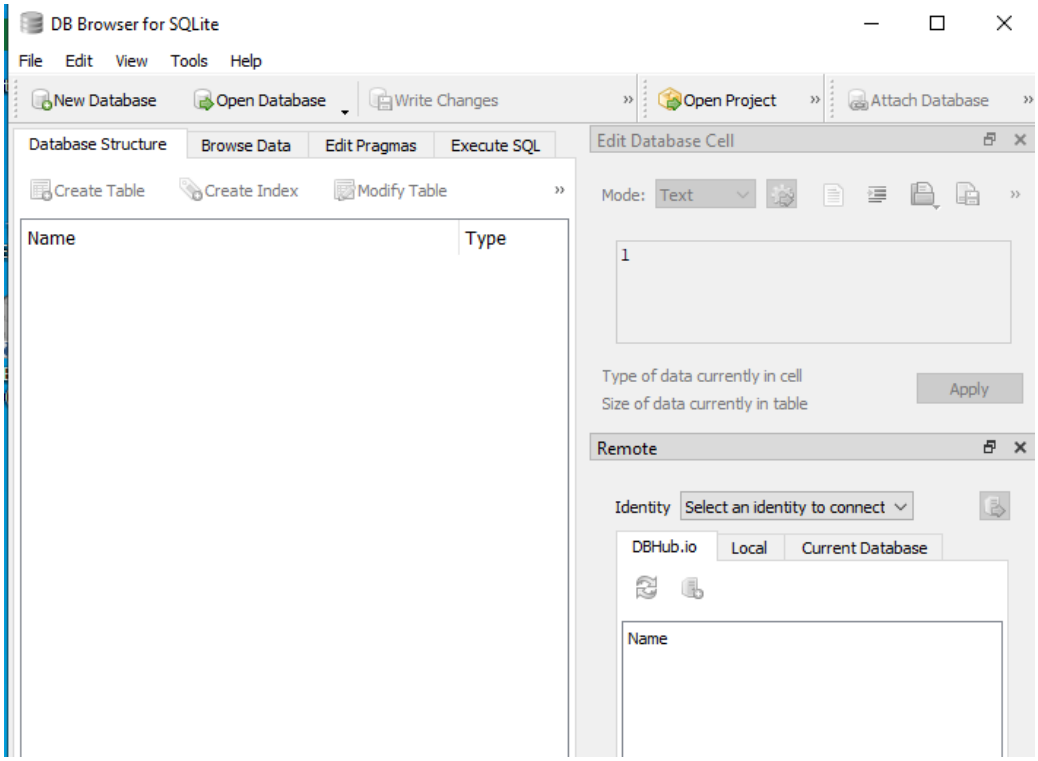
SQLite es **Open Source** y se ha instalado por defecto con Python, es decir forma parte de la biblioteca estándar, no tenemos que instalar ningún módulo con pip.

Crear la base de datos

Podemos crear la base de datos desde visual studio code, previa instalación de sqlite en el sistema operativo y la extensión para conectarse a bases de datos desde visual studio code. La otra forma es utilizando una herramienta visual como **DBBrowser Sqlite**, la cual se puede descargar desde:



Interfaz de la herramienta DBBrowser Sqlite:



Códificación en Python de una aplicación que realice las operaciones del crud a la base de datos creada.

Código ejemplo para conectarnos a la base de datos

```
1   import sqlite3
2
3   baseDatos = "tienda.db"
4   # crear la conexión a la base datos
5   miConexion = sqlite3.connect(baseDatos)
```

Proceso de insertar un Producto a la base de datos

```
7   def agregar():
8       try:
9           codigo = 11
10          nombre = "Nevera"
11          precio = 2000000
12          categoria = "Electrodomestico"
13          #creando una tupla con los datos a insertar
14          producto = (codigo, nombre, precio, categoria)
15          #texto sql de la consulta, se utiliza ? para parámetros
16          consulta = "insert into productos values (null,?,?,?,?)"
17          #crear objeto tipo cursor. Necesario para hacer operaciones en la BD
18          cursor = miConexion.cursor()
19          #El cursor ejecuta la consulta a la base de datos
20          resultado = cursor.execute(consulta, producto)
21          #Se utiliza commit para consultas que modifican la BD.
22          #Insertar, actualizar y eliminar
23          miConexion.commit()
24          if resultado.rowcount == 1:
25              print(f"Producto agregado correctamente con \
26                    id {resultado.lastrowid}")
27      except miConexion.Error as error:
28          miConexion.rollback()
29          print(f"Error: {error}")
```

El código anterior se ha colocado dentro de un ***try-except*** para poder capturar excepciones e identificar si hay algún inconveniente. Por ejemplo, si estamos insertando un producto con un código ya existente se generaría una excepción teniendo en cuenta que el código del producto es un campo único.

Siempre que se vaya a realizar una consulta a la base de datos tenemos que tener un objeto con la conexión, en este ejemplo se llama ***miConexion***; también se debe crear un objeto de tipo ***cursor*** quien es el que nos permite hacer operaciones a la base de datos.

En este caso especial que es insertar un producto, debemos tener los datos del producto en diferentes variables o en una sola variable como está el ejemplo, donde se creó una ***tupla*** con el producto como se muestra en la **línea 14**; también se debe crear el texto de la consulta con o sin parámetros; en el ejemplo se está utilizando parámetros utilizando **?** indicando que ahí va un valor.

En la ***línea 20*** se ejecuta la consulta utilizando el ***cursor*** donde llama el método ***execute*** y se le pasa como argumentos el texto de la ***consulta*** y los datos que contienen los valores

de los parámetros; es importante tener en cuenta que la tupla debe ser creada en el mismo orden como están los parámetros en el texto de la consulta.

Cuando se hagan operaciones de insertar, actualizar y eliminar se debe hacer un ***commit*** para que se acepten los cambios en la base de datos como se muestra en la ***línea 23***. Si hay problemas en la consulta y se genera una excepción, se debe hacer un ***rollback***.

Salida ejemplo insertar producto: En el ejemplo se piden los datos por teclado.

Proceso consultar un producto por su código:

```
32  def consultarPorCodigo():
33      try:
34          codigo = (11,)
35          consulta = "select * from productos where proCodigo=?"
36          cursor = miConexion.cursor()
37          cursor.execute(consulta, codigo)
38          producto = cursor.fetchone()
39          if (producto):
40              print(producto)
41          else:
42              print(f"No existe producto con el código: {codigo[0]}")
43      except miConexion.Error as error:
44          print(f"Error: {error}")
```

En las consultas de tipo ***select*** podemos utilizar el método ***fetchone***() en el ***cursor*** para cuando esperamos solo un registro, y para cuando la consulta puede traer más de un registro utilizamos el método ***fetchall***(). En el código anterior en la ***línea 38*** obtenemos los datos del producto consultado. Los datos de respuesta siempre son almacenados en una tupla.

Salida ejemplo consultar producto:

Proceso actualizar un Producto:

```
59  def actualizar():
60      try:
61          codigo = 11
62          nombre = "Lavadora"
63          precio = 2500000
64          datosActualizar = (nombre, precio, codigo)
65          consulta = "update productos set proNombre=?, proPrecio=? where proCodigo=?"
66          cursor = miConexion.cursor()
67          resultado = cursor.execute(consulta, datosActualizar)
68          miConexion.commit()
69          if (resultado.rowcount == 1):
70              print("Producto actualizado...")
71          else:
72              print("No existe producto con el código.....")
73      except miConexion.Error as error:
74          miConexion.rollback()
75          print(f"Error: {error}")
```

En el código anterior se van actualizar los campos nombre y precio del producto identificado con código 11. Aquí se debe crear una tupla con los campos en el mismo orden como van a aparecer en el texto de la consulta.Como es una consulta de tipo ***update*** se debe hacer ***commit*** para que se acepten los cambios y un ***rollback*** si se va por una excepción.

Salida ejemplo actualizar producto: se piden los datos por teclado y se va solicitando que quiere actualizar.

Proceso eliminar un producto:

```
78  def eliminar():
79      try:
80          codigo = (11,)
81          consulta = "delete from productos where proCodigo=?"
82          cursor = miConexion.cursor()
83          resultado = cursor.execute(consulta, codigo)
84          miConexion.commit()
85          if resultado.rowcount == 1:
86              print("Producto eliminado")
87          else:
88              print("no existe producto con ese código")
89      except miConexion.Error as error:
90          print(f"Error: {error}")
```

En las consultas de tipo ***delete*** se debe hacer el ***commit*** para que se acepten los cambios, igual que en las de tipo ***insert*** y ***update***.

Ejemplo salida eliminar producto

Proceso de obtener todos los productos

```
47  def obtenerProductos():
48      try:
49          consulta = "select * from productos"
50          cursor = miConexion.cursor()
51          cursor.execute(consulta)
52          productos = cursor.fetchall()
53          if (productos):
54              print(productos)
55      except miConexion.Error as error:
56          print(f"Error: {error}")
```

En la consulta anterior como se espera más de un registro utilizamos el método **fetchall** del cursor para obtener todos los productos que arroje la consulta como se muestra en la ***línea 52.***

Ejemplo salida listar todos los productos

Uso de Commit:

Cada vez que usamos la función **execute**() comienza una **nueva transacción** a la base de datos que debe confirmarse con commit() o bien deshacerse con **rollback**().

¿Qué es un Cursor?

El objeto **cursor en Python** es una herramienta poderosa que nos permite interactuar con bases de datos. Gracias a él, podemos ejecutar consultas, insertar datos, actualizar registros y más.

¿Qué es conexión rollback?

El comando de la conexión **rollback** forma parte de los módulos del trabajo con base de datos de **SQL**. Es un módulo que pertenece a la librería sql3 de Python, que controla el paso de conexión entre el programa de código y las tablas o bases de datos.

Por tanto, al hablar de qué es conexión rollback podemos decir que ***se encarga de regresar al lugar del desarrollo del programa donde se realizó el último* commit *de guardado***. Por ejemplo, si se realizan varias operaciones dentro del programa, estas se guardan en un espacio que no ha sido almacenado aún y, si todo sale mal o hay muchos problemas con estos cambios, el comando connection.rollback () regresa al inicio del programa, antes de la ejecución de estos cambios. De esta manera, no se deja ninguna operación a medias.

Referencias:

- Tutorial Python: https://www.w3schools.com/python
- Documentación oficial Python: https://docs.python.org/es/3/library/sqlite3.html
- https://www.tutorialspoint.com/python3/python_database_access.htm
- Tutorial: https://www.tutorialesprogramacionya.com/pythonya/detalleconcepto.php?punto=87&codigo=88&inicio=75
- https://aprendepython.es/stdlib/data_access/sqlite/

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	26-08-2024